

# Memorial Técnico

## 1. Identificação do projeto

**Projeto:** Portal de Solicitações Internas

**Desenvolvedor:** Luiz Inácio Moura da Costa

O projeto foi desenvolvido como parte de um desafio técnico para uma vaga de Desenvolvedor Full Stack Júnior.

O objetivo foi criar uma aplicação web para registrar e acompanhar solicitações internas de uma empresa, permitindo autenticação de usuários, criação e gerenciamento de solicitações, filtros e visualização de indicadores.

## 2. Tecnologias utilizadas

### Frontend

- React
- Vite
- React Router DOM
- Axios
- CSS

### Backend

- Node.js
- Express
- JSON Web Token (JWT)
- bcryptjs

### Banco de dados

- SQLite

## 3. Justificativa das tecnologias

### React

O React foi escolhido para desenvolver a interface por permitir a criação da aplicação a partir de componentes reutilizáveis.

Também facilitou o gerenciamento dos dados exibidos nas páginas utilizando recursos como useState e useEffect.

### Vite

O Vite foi utilizado para criar e executar o projeto React por possuir uma configuração simples e um ambiente de desenvolvimento rápido.

### React Router DOM

Foi utilizado para controlar a navegação entre as páginas da aplicação, como login, dashboard, listagem, criação, detalhes e edição de solicitações.

Também foi utilizado na criação de rotas protegidas para evitar que usuários não autenticados acessem diretamente as páginas internas.

### **Axios**

O Axios foi escolhido para realizar a comunicação entre o frontend e a API do backend.

Com ele são realizadas as requisições de login, criação, consulta, edição, exclusão e alteração de status das solicitações.

### **Node.js e Express**

O backend foi desenvolvido com Node.js e Express.

O Express foi utilizado para criar as rotas da API e organizar o tratamento das requisições HTTP.

A escolha foi feita principalmente pela simplicidade de integração com o frontend em JavaScript e pela facilidade de criar uma API REST.

### **SQLite**

O SQLite foi escolhido por ser um banco de dados simples e adequado ao tamanho do projeto.

Ele não exige a instalação de um servidor separado e permite que o banco seja criado automaticamente pela própria aplicação.

Para um projeto de pequeno porte e execução local, isso facilitou a configuração e os testes.

### **JWT**

O JSON Web Token é utilizado para controlar a autenticação do usuário.

Depois de realizar o login com sucesso, o backend gera um token que deve ser enviado nas requisições das rotas protegidas.

### **bcryptjs**

O bcryptjs é utilizado para gerar o hash da senha do usuário.

Dessa forma, a senha não é armazenada diretamente em texto puro no banco de dados.

## **4. Organização da aplicação**

O projeto foi separado principalmente em frontend, backend, banco de dados e documentação.

```
portal-solicitacoes/  
├── backend/  
├── database/  
├── docs/  
├── frontend/  
└── README.md
```

Essa separação foi utilizada para deixar mais claro o papel de cada parte do sistema.

## 5. Organização do backend

O backend está organizado nas seguintes pastas:

```
backend/src/  
├── banco/  
├── controladores/  
├── middlewares/  
├── rotas/  
└── servidor.js
```

### **banco**

Contém os arquivos responsáveis pela conexão com o SQLite e pela criação do usuário inicial utilizado para demonstração.

### **controladores**

Contém a lógica utilizada pelas rotas.

Foram separados controladores para:

- autenticação;
- solicitações;
- indicadores do dashboard.

### **middlewares**

Contém o middleware responsável pela verificação do token JWT.

Antes de permitir o acesso às rotas protegidas, o middleware verifica se o token enviado é válido.

### **rotas**

Contém a definição dos endpoints da API.

As rotas foram separadas de acordo com a funcionalidade:

- autenticação;
- solicitações;
- dashboard.

### **servidor.js**

É o ponto de inicialização do backend.

Nesse arquivo são configurados o Express, CORS, leitura de JSON, conexão com o banco e registro das rotas da aplicação.

## 6. Organização do frontend

O frontend está organizado principalmente em:

```
frontend/src/  
├── componentes/  
├── paginas/  
├── servicos/  
├── App.jsx  
└── main.jsx
```

## **componentes**

Contém componentes utilizados em diferentes partes da aplicação.

Entre eles estão o cabeçalho e o componente de rota protegida.

## **paginas**

Contém as principais telas do sistema:

- Login;
- Dashboard;
- Solicitações;
- Nova Solicitação;
- Detalhes da Solicitação;
- Editar Solicitação.

## **servicos**

Contém funcionalidades utilizadas por diferentes páginas.

Nesse local fica a configuração do Axios para comunicação com a API e a função utilizada para formatação das datas.

## **App.jsx**

Contém a definição das rotas da aplicação.

As páginas internas são protegidas pelo componente RotaProtegida.

## **7. Modelagem do banco de dados**

O banco possui duas tabelas principais:

- usuarios;
- solicitacoes.

A tabela usuarios armazena os dados necessários para autenticação.

A tabela solicitacoes armazena os dados das solicitações criadas no sistema.

Existe um relacionamento entre as duas tabelas através do campo:

solicitacoes.usuario\_id

que referencia:

usuarios.id

Dessa forma, cada solicitação fica relacionada ao usuário que realizou seu cadastro.

Os detalhes dos campos estão disponíveis no arquivo:

docs/dicionario-de-dados.md

## **8. Autenticação**

O acesso às funcionalidades internas depende de autenticação.

O processo funciona da seguinte forma:

1. O usuário informa seu usuário e senha na tela de login.

2. O frontend envia os dados para o backend.
3. O backend procura o usuário no banco.
4. A senha informada é comparada com o hash armazenado utilizando bcrypt.
5. Caso os dados estejam corretos, o backend gera um token JWT.
6. O frontend armazena o token e o envia nas próximas requisições.
7. O middleware do backend verifica o token antes de permitir o acesso às rotas protegidas.

O token possui tempo de expiração configurado no backend.

O frontend também utiliza uma rota protegida para impedir a navegação normal para páginas internas quando não existe um token armazenado.

A validação realizada pelo backend é a responsável pela proteção efetiva da API.

## 9. Comunicação entre frontend e backend

A comunicação é realizada através de uma API HTTP.

O frontend utiliza Axios para fazer as requisições para:

`http://localhost:3000/api`

Entre as principais operações estão:

```
POST /api/autenticacao/login
GET  /api/solicitacoes
POST /api/solicitacoes
GET  /api/solicitacoes/:id
PUT  /api/solicitacoes/:id
DELETE /api/solicitacoes/:id
PATCH /api/solicitacoes/:id/status
GET  /api/dashboard
```

As rotas de solicitações e dashboard exigem autenticação.

## 10. Regras das solicitações

Uma nova solicitação possui:

- título;
- descrição;
- categoria;
- usuário responsável pela criação;
- data de criação automática;
- status inicial Aberto.

As categorias disponíveis são:

- TI;
- RH;
- Compras;
- Financeiro;
- Infraestrutura.

Os status utilizados são:

- Aberto;

- Em Atendimento;
- Concluído.

Solicitações com status Aberto podem ser editadas ou excluídas.

A alteração de status é feita separadamente através de uma rota específica.

## **11. Filtros**

A listagem de solicitações permite a utilização de filtros por:

- título;
- categoria;
- status;
- período.

Os filtros são enviados pelo frontend através dos parâmetros da requisição.

No backend, a consulta SQL é montada de acordo com os filtros recebidos.

Foram utilizados parâmetros na consulta para evitar a inserção direta dos valores enviados pelo usuário no SQL.

## **12. Dashboard**

O dashboard apresenta quatro indicadores:

- total de solicitações;
- solicitações abertas;
- solicitações em atendimento;
- solicitações concluídas.

Os valores são calculados diretamente no banco de dados e retornados pelo backend.

## **13. Tratamento de erros e validações**

Foram adicionadas validações tanto no frontend quanto no backend.

O frontend utiliza campos obrigatórios nos formulários e exibe mensagens quando uma operação não pode ser concluída.

O backend também verifica os dados recebidos antes de realizar alterações no banco.

Alguns exemplos são:

- usuário ou senha não informados;
- categoria inválida;
- status inválido;
- solicitação inexistente;
- tentativa de editar uma solicitação que não está aberta;
- tentativa de excluir uma solicitação que não está aberta;
- acesso a uma rota protegida sem um token válido.

As validações do backend são mantidas mesmo quando existem validações equivalentes no frontend, pois as requisições para a API também podem ser realizadas sem utilizar a interface.

## **14. Responsividade**

A interface recebeu ajustes para funcionar em diferentes tamanhos de tela.

No dashboard, a quantidade de colunas dos indicadores é reduzida conforme a largura disponível.

Os formulários e botões também se adaptam a telas menores.

Na listagem de solicitações, a tabela utiliza rolagem horizontal quando não existe espaço suficiente para mostrar todas as colunas.

## **15. Limitações atuais**

Por se tratar de um projeto desenvolvido para um desafio técnico e com escopo reduzido, algumas funcionalidades não foram implementadas.

Entre as principais limitações estão:

- não existe tela para cadastro de novos usuários;
- o usuário de demonstração é criado através de um script;
- não existem diferentes níveis ou perfis de acesso;
- não existe paginação na listagem de solicitações;
- não foram implementados testes automatizados;
- o banco SQLite é local;
- não existe implantação em ambiente de produção.

Atualmente, as operações são autorizadas para usuários autenticados, enquanto as regras de edição e exclusão são controladas principalmente pelo status da solicitação.

## **16. Melhorias futuras**

Como evolução do sistema, poderiam ser adicionados:

- cadastro e gerenciamento de usuários;
- diferentes perfis de acesso;
- restrição de edição e exclusão de acordo com o usuário responsável;
- paginação da listagem;
- histórico das alterações de status;
- testes automatizados de frontend e backend;
- recuperação de senha;
- logs de operações;
- notificações;
- implantação da aplicação em ambiente de produção.

## **17. Diferenças para um ambiente de produção**

Em um ambiente real de produção, algumas decisões técnicas seriam revistas.

O SQLite poderia ser substituído por um banco de dados como PostgreSQL ou MySQL, principalmente em um cenário com vários usuários acessando o sistema ao mesmo tempo.

A aplicação também deveria utilizar HTTPS e uma configuração mais segura para armazenamento e envio do token de autenticação.

A chave utilizada para gerar os tokens JWT deveria ser configurada através de uma variável de ambiente segura e possuir um valor forte.

Também seriam importantes recursos como:

- controle de permissões;
- logs;
- monitoramento;
- tratamento centralizado de erros;
- testes automatizados;
- proteção contra excesso de requisições;
- configuração específica de CORS;
- processo automatizado de deploy.

## **18. Conclusão**

O projeto atende às principais funcionalidades propostas para o Portal de Solicitações Internas.

Durante o desenvolvimento foi possível trabalhar com a integração entre frontend, backend e banco de dados, além de autenticação, rotas protegidas, operações CRUD, filtros e construção de uma interface responsiva.

A solução foi mantida simples, priorizando a funcionalidade, organização do código e facilidade de execução do projeto.